

```
1 #===== 训练 lec3 的 MLP 并采样生成名字 (支持GPU加速) =====
2 # (1) 为什么不直接 import lec3_AdjustParameters_part2?
3 #   1.那个文件在模块顶层就直接跑训练循环、画图,还调用了ipywidgets的interact,
4 #   一旦import就会把整套东西重新执行一遍 (而且会卡在widget上)
5 #   2.它的目录名 "Neural Network_Zero to hero"带空格和横杠,本身也不是合法的包名
6 #   3.所以这里把数据集准备和几个layer类重新写了一遍,结构、超参、随机种子都和lec3保持一致
7 # (2) 相对lec3,本文件做了四处改动:
8 #   1.所有参数和数据都搬到 device 上 (cuda / mps / cpu 自动选择)
9 #   2.去掉了训练时的调试开销: layer.out.retain_grad() 和 ud 统计,它们会明显拖慢训练
10 #   3.采样函数 use() 自己负责切换BatchNorm的eval/training模式,并在结束时还原
11 #   4.默认的batch_size从32改成256、步数从20万改成2.5万,两者相乘都是640万个样本,
12 #   看过的数据量不变,但每次更新用256个样本、梯度更稳,步数只要1/8,跑起来快得多
13 #   (想完全复现lec3的动态: --batch_size 32 --max_steps 200000)
14 # (3) 本文件不会修改任何lec3的文件
15 # (4) 实测性能 (RTX 4070 Laptop, batch_size=32, n_hidden=1000, 共420万个参数):
16 #   1.每一步约4.4ms: 前向1.35ms + 反向2.56ms + 参数更新0.50ms
17 #   2.但每一步的计算量只有约0.8GFLOP,按这张卡的算力只要0.2ms左右就能算完,
18 #   说明瓶颈不在计算,而在"算子数量太多、每个kernel的启动与调度开销"
19 #   3.这也解释了为什么换到GPU只比CPU快两倍以上: batch只有32、宽度只有1000,单次计算实在太小
20 #   4.想再快,要么减少kernel数量 (torch.compile + inductor,
21 #   但Windows上需要额外安装triton-windows,实测默认后端会报TritonMissing),
22 #   要么加大batch_size: 本文件默认已经这么做了 (见上面第4条改动),
23 #   batch从32加到256以后,单步里那些固定开销被摊薄,整体快了不止两倍
24 import argparse
25 import random
26 import time
27 from pathlib import Path
28
29 import torch
30 import torch.nn.functional as F
31
32
33 #===== 命令行参数 =====
34 def parse_arguments():
35     parser = argparse.ArgumentParser(description='训练lec3的MLP并采样生成名字')
36     # (1) 默认配置: batch_size=256、max_steps=25000
37     #   两者相乘是640万个样本.和lec3 (batch 32 x 20万步) 看过的样本总量一致.
```

```
38 # 好处是每次更新用256个样本、梯度更稳，步数只需要上一版的1/8，跑得快很多
39 # 代价是更新次数变少，训练动态不再和lec3逐位等价
40 parser.add_argument('--max_steps', type=int, default=25000,
41                     help='训练步数(默认25000,和batch_size=256配套)')
42 parser.add_argument('--batch_size', type=int, default=256,
43                     help='每个minibatch的样本数(默认256;想复现lec3就设32并配--max_steps 200000)')
44 parser.add_argument('--number_of_names', type=int, default=10,
45                     help='交互式采样里“直接回车”时生成的名字个数(默认10)')
46 parser.add_argument('--device', type=str, default='auto',
47                     help='训练设备:auto / cuda / mps / cpu(默认auto)')
48 return parser.parse_args()
49
50
51 arguments = parse_arguments()
52
53
54 #===== 设备选择 =====
55 def choose_device(requested_device):
56     if requested_device != 'auto':
57         return torch.device(requested_device)
58     if torch.cuda.is_available():                # NVIDIA显卡
59         return torch.device('cuda')
60     if torch.backends.mps.is_available():        # 苹果芯片的GPU
61         return torch.device('mps')
62     return torch.device('cpu')
63
64
65 device = choose_device(arguments.device)
66 print(f'使用设备: {device}')
67 if device.type == 'cuda':
68     print(f'显卡型号: {torch.cuda.get_device_name(0)}')
69
70
71 #===== 数据集准备 =====
72 # (1) 用__file__来定位names.txt: 这样不管在哪个目录下运行脚本, 都能找到数据文件
73 data_path = Path(__file__).resolve().parent / 'names.txt'
74 words = data_path.read_text(encoding='utf-8').splitlines()
```

```
75
76 chars = sorted(list(set(''.join(words))))
77 stoi = {s:i+1 for i,s in enumerate(chars)}
78 stoi['.'] = 0
79 itos = {i:s for s,i in stoi.items()}
80 vocab_size = len(itos)
81 print(itos)
82 print(vocab_size)
83
84 block_size = 5 # 上下文长度: 读入多少个上文, 来预测下一个char
85
86
87 def build_dataset(words):
88     X, Y = [], []
89
90     for w in words:
91         context = [0] * block_size
92         for ch in w + '.':
93             ix = stoi[ch]
94             X.append(context)
95             Y.append(ix)
96             context = context[1:] + [ix] # crop and append
97
98     X = torch.tensor(X)
99     Y = torch.tensor(Y)
100     print(X.shape, Y.shape)
101     return X, Y
102
103
104 # (2) 和lec3用同样的随机种子, 保证切分出来的训练/验证/测试集完全一致
105 random.seed(42)
106 random.shuffle(words)
107 n1 = int(0.8*len(words))
108 n2 = int(0.9*len(words))
109
110 Xtr, Ytr = build_dataset(words[:n1]) # 80%
111 Xdev, Ydev = build_dataset(words[n1:n2]) # 10%
```

```
112 Xte, Yte = build_dataset(words[n2:])      # 10%
113
114
115 #===== 线性层 =====
116 class Linear:
117
118     def __init__(self, fan_in, fan_out, bias=True):
119         self.weight = torch.randn((fan_in, fan_out), generator=g) / fan_in**0.5
120     # 良好的初始化Weight矩阵, 确保数值分布合理
121         self.bias = torch.zeros(fan_out) if bias else None
122     # 一开始不设置任何偏置———我们希望线性层的输出, 大致符合标准正态分布
123
124     def __call__(self, x):
125         self.out = x @ self.weight
126         if self.bias is not None:
127             self.out += self.bias
128         return self.out
129
130     def parameters(self):
131         return [self.weight] + ([self.bias] if self.bias is not None else [])
132
133
134 #===== BatchNorm层 =====
135 class BatchNorm1d:
136
137     def __init__(self, dim, eps=1e-5, momentum=0.1):
138         self.eps = eps
139         self.momentum = momentum
140         self.training = True
141
142         # parameters (trained with backprop)
143         self.gamma = torch.ones(dim)
144         self.beta = torch.zeros(dim)
145
146         # buffers (trained with a running 'momentum update')
147         self.running_mean = torch.zeros(dim)
148         self.running_var = torch.ones(dim)
```

```
149
150 def __call__(self, x):
151     if self.training:
152         xmean = x.mean(0, keepdim=True) # batch mean
153         xvar = x.var(0, keepdim=True)   # batch variance
154     else:
155         xmean = self.running_mean
156         xvar = self.running_var
157     xhat = (x - xmean) / torch.sqrt(xvar + self.eps)
158     self.out = self.gamma * xhat + self.beta
159
160     #在训练的过程中不断更新逼近全局mean
161     if self.training:
162         with torch.no_grad():
163             self.running_mean = (1 - self.momentum) * self.running_mean + self.momentum * xmean
164             self.running_var = (1 - self.momentum) * self.running_var + self.momentum * xvar
165     return self.out
166
167 def parameters(self):
168     return [self.gamma, self.beta]
169
170
171 #===== 激活函数层 =====
172 class Tanh:
173     def __call__(self, x):
174         self.out = torch.tanh(x)
175         return self.out
176     def parameters(self):
177         return []
178
179
180 #===== 初始化参数 =====
181 n_embd = 30      # 字符嵌入向量的维度
182 n_hidden = 1000 # 隐藏层神经元个数
183 g = torch.Generator().manual_seed(2147483647) # for reproducibility
184
185 C = torch.randn((vocab_size, n_embd), generator=g)
```

```
186 layers = [  
187     Linear(n_embd * block_size, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),  
188     Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),  
189     Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),  
190     Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),  
191     Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),  
192     Linear(n_hidden, vocab_size, bias=False), BatchNorm1d(vocab_size),  
193 ]  
194  
195 # (1) 最后一层BN的gamma乘0.1: 让网络一开始输出接近均匀分布, 而不是"自信地瞎猜"  
196 # 所以初始loss大约是3.29, 而不是没做这个压制时的3.80  
197 # 注: lec3里还有一段 layer.weight *= 1.0, 那是保留作警示的空操作, 这里就不写了  
198 with torch.no_grad():  
199     layers[-1].gamma *= 0.1  
200  
201 # (2) 把嵌入矩阵、所有层的参数、BatchNorm的running统计量都搬到device上  
202 # 必须在设置requires_grad之前搬: 已经是叶子且要梯度的张量, 不好再做in-place替换  
203 C = C.to(device)  
204 for layer in layers:  
205     if isinstance(layer, Linear):  
206         layer.weight = layer.weight.to(device)  
207         if layer.bias is not None:  
208             layer.bias = layer.bias.to(device)  
209     elif isinstance(layer, BatchNorm1d):  
210         layer.gamma = layer.gamma.to(device)  
211         layer.beta = layer.beta.to(device)  
212         #running_mean / running_var不是参数, 但eval模式和采样都要用到, 也必须搬过去  
213         layer.running_mean = layer.running_mean.to(device)  
214         layer.running_var = layer.running_var.to(device)  
215  
216 # (3) 把所有参数收集到一个列表里, 统一做梯度清零和更新  
217 parameters = [C] + [p for layer in layers for p in layer.parameters()]  
218 print('参数总量:', sum(p.nelement() for p in parameters))  
219 for p in parameters:  
220     p.requires_grad = True  
221  
222 # (4) 数据集也搬到device上. 这样每个step就不需要反复做CPU->GPU的拷贝
```

```
223 Xtr, Ytr = Xtr.to(device), Ytr.to(device)
224 Xdev, Ydev = Xdev.to(device), Ydev.to(device)
225 Xte, Yte = Xte.to(device), Yte.to(device)
226
227
228 #===== 开始训练 =====
229 batch_size = arguments.batch_size
230 index_chunk_size = 1024 # 一块索引够用多少步 (默认256*1024个int64, 约占2MB显存)
231 lossi = []
232 start_time = time.time()
233
234 for step in range(arguments.max_steps):
235
236     # (1) 取一个minibatch
237     # 1.如果每一步都做一次 torch.randint(...).to(device), 就等于每步做一次CPU->GPU拷贝,
238     # 而普通内存(pageable)的拷贝会同步整条CUDA流, CPU没办法提前把后面的kernel排进队列,
239     # 实测这一步会让训练慢好几倍
240     # 2.所以改成"整块抽、整块搬、按步切": 每index_chunk_size步才做一次拷贝,
241     # 其余步骤只是从GPU上的这块索引里切一片 (切片是view, 既不拷贝也不同步)
242     # 3.为什么不影响复现性: 这个generator是确定性的,
243     # "一次抽N*t个"和"分t次每次抽N个"得到的序列完全相同 (已实试验证),
244     # 所以minibatch的选取顺序和lec3一模一样
245     if step % index_chunk_size == 0:
246         chunk_steps = min(index_chunk_size, arguments.max_steps - step)
247         index_chunk = torch.randint(0, Xtr.shape[0], (chunk_steps * batch_size,), generator=g).to(device)
248
249         chunk_offset = (step % index_chunk_size) * batch_size
250         ix = index_chunk[chunk_offset : chunk_offset + batch_size]
251         Xb, Yb = Xtr[ix], Ytr[ix]
252
253     # (2) 前向传播
254     emb = C[Xb]
255     x = emb.view(emb.shape[0], -1)
256     for layer in layers:
257         x = layer(x)
258     loss = F.cross_entropy(x, Yb)
259
```

```
260 # (3) 反向传播
261 for p in parameters:
262     p.grad = None
263 loss.backward()
264
265 # (4) 更新参数: 训练后1/4的阶段把学习率降到1/10 (和lec3的150000/200000一致)
266 # 注: batch变大后理论上的经验做法是按比例调大学习率, 但这里故意保持lec3的0.1不动,
267 # 因为它对256的batch依然是安全的, 少引入一个变量
268 learning_rate = 0.1 if step < 0.75 * arguments.max_steps else 0.01
269 for p in parameters:
270     p.data += -learning_rate * p.grad
271
272 # (5) 记录loss, 并且定期打印进度和耗时
273 # 注意: 不要在每一步都调用 .item()!
274 # .item() 会把GPU的异步执行强行同步: CPU必须等GPU把这一步算完才能继续,
275 # 于是每一次kernel启动的开销都被完整暴露出来 (实测能慢三倍以上)
276 # 所以这里每100步才同步一次, 用来记录loss曲线
277 if step % 100 == 0 or step == arguments.max_steps - 1:
278     lossi.append(loss.item())
279
280 if step % 10000 == 0 or step == arguments.max_steps - 1:
281     elapsed = time.time() - start_time
282     print(f'{step:7d}/{arguments.max_steps:7d}: loss {loss.item():.4f} | 已用时 {elapsed:.1f}s')
283
284
285 #===== 评估train/val/test的loss =====
286 evaluation_batch_size = 4096
287
288 # (1) 把所有BatchNorm层切到指定模式
289 def set_batch_norm_mode(training):
290     for layer in layers:
291         if isinstance(layer, BatchNorm1d):
292             layer.training = training
293
294 # (2) 记录当前每个BatchNorm层的模式, 方便结束后还原
295 def save_batch_norm_modes():
296     return [(layer, layer.training) for layer in layers if isinstance(layer, BatchNorm1d)]
```

```
297
298 # (3) 把之前记录的模式还原回去
299 def restore_batch_norm_modes(saved_modes):
300     for layer, training in saved_modes:
301         layer.training = training
302
303
304 @torch.no_grad()
305 def split_loss(split):
306     x, y = {
307         'train': (Xtr, Ytr),
308         'val': (Xdev, Ydev),
309         'test': (Xte, Yte),
310     }[split]
311
312     # (4) 为什么要分块算? val集有2万多个样本, 一次性喂进去的话,
313     #     5个隐藏层的激活值各占几百MB显存, 会白白吃满显存
314     #     分块时用reduction='sum'再按样本数平均, 结果和一次性算完全相同
315     total_loss = 0.0
316     for start in range(0, x.shape[0], evaluation_batch_size):
317         Xb = x[start:start + evaluation_batch_size]
318         Yb = y[start:start + evaluation_batch_size]
319         emb = C[Xb]
320         out = emb.view(emb.shape[0], -1)
321         for layer in layers:
322             out = layer(out)
323         total_loss += F.cross_entropy(out, Yb, reduction='sum').item()
324
325     average_loss = total_loss / x.shape[0]
326     print(f'{split:5s} loss: {average_loss:.4f}')
327     return average_loss
328
329
330 # (5) 评估必须在eval模式下做: BN要用训练时累积的running统计量,
331 #     否则"整个验证集一起算batch统计量"会把答案泄漏进BN里, 算出来的loss虚低
332 saved_modes = save_batch_norm_modes()
333 set_batch_norm_mode(False)
```



```
371         emb = C[torch.tensor([context], device=device)]
372         x = emb.view(emb.shape[0], -1)
373         for layer in layers:
374             x = layer(x)
375         probabilities = F.softmax(x, dim=1)
376
377         # (5) 把概率搬到CPU再采样:
378         #   torch.multinomial的generator必须和tensor在同一个设备上,
379         #   而固定用CPU上的generator, 可以保证CPU和GPU训出来的模型生成的名字一致
380         character_index = torch.multinomial(probabilities[0].cpu(), num_samples=1,
381                                           generator=sample_generator).item()
382
383         # (6) 抽到 '.' (编号是0) 说明名字结束了: 先判断再append, 所以打印的名字不带 '.'
384         if character_index == 0:
385             break
386
387         # (7) 记录字符, 并把上下文滑动一格
388         generated_characters.append(itos[character_index])
389         context = context[1:] + [character_index]
390
391         # (8) 拼成字符串就是生成出来的一个名字
392         print(''.join(generated_characters))
393     finally:
394         # (9) 不管生成过程有没有出错, 都把BatchNorm模式还原, 不影响调用者
395         restore_batch_norm_modes(saved_modes)
396
397
398 #===== 交互式采样: 输入个数就能生成名字 =====
399 # (1) 训练和评估都做完了以后, 进入一个交互循环:
400 #   输入一个数字, 就生成这么多个名字; 输入no, 就结束程序; 直接回车则用默认个数
401 #   (默认个数由命令行参数--number_of_names决定, 默认是10)
402 def ask_and_generate(default_number_of_names):
403     while True:
404         user_input = input(f'请输入要生成的名字个数 (回车=默认{default_number_of_names}, 输入 no 结束程序) : ').strip()
405
406         # (2) 输入no (不区分大小写, 前面strip过, 所以多余空格也被去掉了) 就结束程序
407         if user_input.lower() == 'no':
```

```
408     print('程序结束')
409     return
410
411     # (3) 直接回车：用命令行参数给的默认个数
412     if user_input == '':
413         number_of_names = default_number_of_names
414
415     # (4) 输入了数字：按用户要的个数生成
416     elif user_input.isdigit():
417         number_of_names = int(user_input)
418
419     # (5) 其他输入：提示一下，然后重新问
420     else:
421         print(f'无法识别"{user_input}"，请输入一个正整数，或者输入 no 结束程序')
422         continue
423
424     if number_of_names <= 0:
425         print('个数要大于0，请重新输入')
426         continue
427
428     print(f'\n生成{number_of_names}个名字：')
429     use(number_of_names)
430
431
432 # (6) 开始交互式采样
433 # 用Ctrl+C也能强制退出：这里捕获KeyboardInterrupt，避免打出一大段报错信息
434 try:
435     ask_and_generate(arguments.number_of_names)
436 except KeyboardInterrupt:
437     print('\n收到 Ctrl+C，程序结束')
```